

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..16.6

SHARED PACKAGES

 Copy as Markdown

Shared packages

Shared packages let you define models and logic that can be safely imported in both server and client code. They contain the models and the protocol file, depend exclusively on the `serverpod_serialization` package, and have no server-only dependencies. This makes them ideal for data structures that need to be used across your full stack, for example DTOs, API request/response shapes, or domain models that flow between Flutter and your Serverpod backend with their custom logic.

Models and the protocol file are generated in the shared package's own directory when your server project's code generation runs. The shared package is tied to the project through the `shared_packages` field in `config/generator.yaml`.

Setup

Create the shared package

Create a new Dart package, for example with `dart create -t package my_shared_package`, and give it a minimal `pubspec.yaml`:

```
name: my_shared_package
description: Models shared between server and client
version: 1.0.0
publish_to: none

environment:
  sdk: ^3.12.2

dependencies:
  serverpod_serialization: SERVERPOD_VERSION
```

 INFO



Use the same Serverpod version as your project. Replace `SERVERPOD_VERSION` with your Serverpod version (for example, `4.0.0`).

Add model files

Place your `.spy.yaml` model files anywhere under the package's `lib` directory:

```
my_shared_package/  
├── lib/  
│   ├── my_shared_package.dart  
│   └── src/  
│       └── models/  
│           └── shared_model.spy.yaml  
└── pubspec.yaml
```

Example model:

```
# lib/src/models/shared_model.spy.yaml  
class: SharedModel  
fields:  
  id: UuidValue, default=random  
  name: String  
  description: String?  
  createdAt: DateTime, default=now
```

Configure the server project

Add the shared package to your server's `config/generator.yaml`:

```
shared_packages:  
  - ../my_shared_package
```

Paths are relative to the server project directory. You can list multiple shared packages.

Generate the code

With `serverpod start` running, saving the model files generates the code. Outside a session, run `serverpod generate` from your server directory.

This generates the Dart classes and protocol in the shared package's `lib/src/generated/` directory. After generation, a typical shared package looks like:

```
my_shared_package/  
├── lib/  
│   ├── my_shared_package.dart  
│   └── src/
```

```

├── generated/
│   ├── protocol.dart
│   └── models/
│       └── shared_model.dart
├── models/
│   └── shared_model.spy.yaml
└── pubspec.yaml

```

Then, add the export for the `protocol.dart` file to the shared package's `lib/my_shared_package.dart` file to make the classes available in the shared package:

```
export 'src/generated/protocol.dart';
```

Add the dependency to the server and client

Add the shared package to both your server and client (or Flutter app) `pubspec.yaml`:

```

# In my_project_server/pubspec.yaml and my_project_client/pubspec.yaml
dependencies:
  my_shared_package:
    path: ../my_shared_package

```

You are now ready to use the shared package in your server and client code.

Using shared models

The shared package is nothing more than a regular Dart package that contains the models and the protocol file.

Importing the package

You can import it in your server and client code just like any other package and use the generated classes:

```

import 'package:my_shared_package/my_shared_package.dart';

// Use in endpoints, Flutter widgets, etc.
final profile = UserProfile(
  displayName: 'Alice',
  avatarUrl: 'https://example.com/avatar.png',
);

```

Table models

A shared model can declare a `table`, as long as it also sets `database: all` (or the experimental `database: sync`). A shared package is used from both the server and the client, so its tables have to be declared for both.

```
class: SharedRecord
table: shared_record
database: all
fields:
  name: String
```

Extending shared models

You can also define a base model in a shared package and extend it on the server, which keeps the table definition out of the shared package.

In the shared package (`lib/src/shared/vehicle.spy.yaml`):

```
class: Vehicle
fields:
  id: UuidValue, default=random
  brand: String
  model: String
```

On the server (`lib/src/models/car.spy.yaml`):

```
class: Car
extends: Vehicle
table: cars
fields:
  year: int
```

The server model extends the shared `Vehicle` and adds a `table` for database persistence plus any additional fields. You can also add server-only fields with `scope=serverOnly` in the server subclass.

Note that the `Car` class will be available in the server and client packages as normal, unless it is defined as `serverOnly`.

Referencing shared models in server models

Shared model names are available in the same namespace when the shared package is configured. Reference them directly in `extends`, `fields`, and other model definitions:

```
class: MyModel
fields:
  sharedModel: SharedModel
  sharedModels: List<SharedModel>
```

Shared packages in a module

A `module` declares its shared packages the same way, in its own `config/generator.yaml`:

```
shared_packages:  
- ../my_module_shared
```

Projects that install the module reach those models through `my_module_server` and `my_module_client`, so they need no dependency on the shared package itself.

Restrictions

Shared models support most Serverpod model features, with these exceptions:

Restriction	Reason
A <code>table</code> requires <code>database:</code> <code>all</code> or <code>sync</code>	The package is used on both sides, so its tables cannot be limited to the server or the client.
No <code>serverOnly</code> on the class	Models must be usable on both server and client.
No <code>scope: serverOnly</code> on fields	All fields must be serializable for the client.

If you need server-only fields, define them in a server model that extends the shared model.

The shared package can also contain custom serializable classes. Register them in the server's `generator.yaml` under `extraClasses` if they need to be used in protocol serialization. See [Custom serializable classes](#) for details.